

FOR EDUCATORS AND AI PRACTITIONERS

Agentic loops: from prompt to autonomous process

A survey of the state of the art, patterns, and a case study you can run yourself

TOPIC	AUTHOR
Agentic loops (agentic AI)	Błażej Strus
CATEGORY	DATE
Loop engineering	July 2026

► IN BRIEF

You probably know the feeling. You ask AI for something, get an almost-good answer, fix it, ask again, and round it goes, all by hand. **An agentic loop** takes that work off your hands. The model plans on its own, reaches for tools, checks its own result and fixes it until it lands. This is not marketing, it is a well-documented engineering pattern. I wrote this with two readers in mind at once: a colleague who teaches this to students, and a practitioner who simply wants their agent to work. I explain it in plain language, show a live Python example, add my own benchmark, and at the end reveal something new, a loop that can improve itself.

WHAT YOU'LL LEARN

- 01** You'll tell a task, a workflow and an agent apart, and recognize when a loop pays off and when it is a trap.
- 02** You'll learn the design patterns (Ng, Anthropic, OpenAI) and their scientific lineage: ReAct, Reflexion, Self-Refine.
- 03** You'll understand why verification must be external, and follow a working example: a non-trivial math problem with a control function in Python.

1 What an agentic loop is

NOTE

i

How to read this. You don't have to follow every paper I cite to get value here. Sections 1–3 give intuition and a glossary, section 4 is a map of sources for the curious, and 5–8 are the concrete part, best shown to students or run yourself from the repository. If you're in a hurry, start with sections 5 and 7, the rest can wait.

Most of us use AI in **one prompt at a time** mode: we type a command, wait, fix, ask again. It works, but we are the engine. When we stop clicking, everything stops. A loop works differently. You give the goal once, and the model carries it out on its own: **it plans, does the work with tools, checks its own result and fixes it**, until the goal is met or a stop condition fires. You step out of the loop, and the work keeps going.

Today we have one simple definition of an agentic loop. Anthropic, after first separating workflows and agents, describes an agent simply as **an LLM autonomously using tools in a loop**. A minimal but precise core, and a good starting intuition.

► DEFINITION • AGENTIC LOOP

A cycle in which a language model plans on its own, takes actions through tools, verifies the result against the goal and iterates, until the goal is met or a stop condition fires. Unlike a single prompt, the human does not steer every step.

NOTE

i

The canonical core of the loop is five phases:

- DISCOVER (what needs doing),
- PLAN (how),
- EXECUTE (do it),
- VERIFY (check against the goal),
- ITERATE (come back with the result).

Three of them – verification, state and the stop condition – decide whether the loop works or just burns tokens.

2 Taxonomy: task, workflow, agent

Before you build a loop, name the problem. Barry Zhang of Anthropic proposes three levels. **A task** is a single model call. **A workflow** is many calls in a control flow that a human designs. In an **agent**, the model owns the flow and decides the next steps itself. The first rule goes: **don't build agents for everything**.

Level	Who controls the flow	When to use
A task	Human (one prompt)	Classification, extraction, summarization
A workflow	Human (code, predefined paths)	A decision tree can be mapped
Agent	Model (decisions at run time)	Ambiguous, high-value task

Zhang's rule of thumb: if you can map a decision tree, build a workflow. You get more accuracy, more control and lower cost than any agent can give. An agent costs more because it explores.

3 Design patterns: a glossary

Two taxonomies organize how we think about loops today. Below I explain each pattern in turn, because they keep coming back in docs and articles as shorthand worth taking literally.

Five workflow patterns (Anthropic)

A workflow is a pre-designed flow of many model calls, steered by a human (code). Anthropic distinguishes five basic layouts.

Prompt chaining – splitting a task into a sequence of steps, where the output of one call is the input to the next. Example: first generate an outline, then expand it into text. Use it when the task has clear, consecutive stages.

Routing – a first model classifies the input and sends it to a specialized path. Example: a ticket goes to returns, technical support or sales. Use it when there are disjoint categories that need different handling.

Parallelization – several calls run in parallel: we either split the task into independent parts, or do the same work several times and vote on the result. Example: three independent reviews of a text and an aggregation. Use it for speed or higher confidence.

Orchestrator-workers – a central model dynamically splits the task, delegates subtasks to workers and merges the results. Unlike parallelization, the split is not fixed up front but emerges as it goes. Example: a complex change spread across many code files.

Evaluator-optimizer – one model generates a result, a second scores it and returns feedback, and so on in a loop until quality is enough. This is a loop in its purest form. Use it when you have clear evaluation criteria and iterative polishing really improves the result.

Four agentic patterns (Ng)

Andrew Ng named four patterns that most often raise the quality of agentic systems. They can be combined in one system.

Reflection – the model critiques its own result and improves it on the next pass, like an automatic reviewer. Note: mostly effective when the critique rests on an external signal (see section 5).

Tool use – the model reaches beyond its own knowledge: it calls APIs, runs code, reads databases. This turns a text generator into a system that actually does things.

Planning – the model breaks the goal into steps and adapts the plan when a step fails. It lets you handle multi-step tasks instead of lunging straight at the solution.

Multi-agent collaboration – several specialized agents share the work (e.g. author and reviewer). This is where the maker/checker pattern is born, the key to verification.

4 Scientific lineage: methods worth knowing

Popular texts talk about "the loop" without names. So I gathered the peer-reviewed foundations in one place, so you don't have to hunt for them, and described what each method solved and how it works. I don't assume you've read every one of these papers, relax. The starting point is how we get a model to reason step by step at all.

Chain-of-Thought (Wei et al., 2022) – prompting the model to write out successive reasoning steps, instead of giving the answer straight away, clearly improves accuracy on multi-step tasks. It is the foundation the later methods build on.

ReAct (Yao et al., 2022) – interleaves reasoning with action in a Thought → Action → Observation cycle. The model alternates between writing a reasoning step and taking a tool action (e.g. a search), and pulls the observation from the environment back into its reasoning. Contact with "ground truth" limits the hallucinations typical of chain-of-thought alone. It is the core of the PLAN/EXECUTE pair.

Reflexion (Shinn et al., 2023) – after a failed attempt the model writes a verbal self-reflection ("why it failed") into episodic memory and uses it on the next try, without retraining weights, in language alone. It is the scientific lineage of the VERIFY and STATE phases: learning from failures between attempts.

Self-Refine (Madaan et al., 2023) – one model in three roles: it generates a result, gives itself feedback, improves, and repeats until the criteria are met. It shows that iterative polishing helps, but (as we'll see) only when the feedback is accurate.

CRITIC (Gou et al., 2024) – a step further: self-correction not from introspection but through external tools (an interpreter, a search engine, a verifier). The model confronts its result with an objective tool and fixes it. It is the direct lineage of the "external gate" in our case study.

Chain-of-Verification (Dhuliawala et al., 2023) – the model first writes a draft, then composes check questions, answers them independently (so as not to be swayed by the draft) and finally edits a verified version. It lowers hallucinations because it separates producing from checking even within one model.

Tree of Thoughts (Yao et al., 2023) – instead of a single reasoning path the model considers many branches, scores them and searches like a decision tree. It is an extension of the planning and search pattern.

These methods map straight onto the loop's phases: Chain-of-Thought and ReAct onto PLAN/EXECUTE, Reflexion onto STATE, and Self-Refine, CRITIC and CoVe onto VERIFY. The shared conclusion leads to section 5: the strongest verification is external.

5 The heart of it: verification

Verification is the heart of the loop. Without a real check of the result there is no loop, just a model that keeps agreeing with itself. Some methods from section 4 (Self-Refine, reflection) assume intrinsic self-correction: the model fixes its own answer solely on the basis of its own judgment, with no external signal. And here comes the most important scientific finding that popular texts leave out.

✗	PITFALL Anti-pattern: the model judges its own work. Huang et al. (ICLR 2024) showed that without external feedback LLMs do not improve their own reasoning, and the result after self-correction can get worse. Stechly, Valmeekam and Kambhampati (2024) confirm the limits of self-verification in reasoning and planning. A model that did the work is too lenient a judge.
✓	BEST PRACTICE The verifying gate must be external and objective: a test, a typecheck, a build, a validator or a hard numeric condition. Separate the doer from the checker (the maker/checker pattern) and make the agent show proof – the test result, the command and its output – instead of declaring success.

This justifies both things at once: the claim that "verify is the heart of the loop" and the pattern of separating doer and reviewer. Coding is a flagship agentic case precisely because unit tests provide cheap, external verification.

6 Costs and silent failures

Loops run on tokens, and cost does not grow linearly, because at each iteration the context sent to the model again grows too. Anthropic gives hard numbers from its own deployments.

Mode	Token usage	Takeaway
Single chat	×1 (baseline)	Cheap, predictable
Agent in a loop	≈ ×4	Justified with real autonomy
Multi-agent system	≈ ×15	High-value tasks only

!	CAUTION The metric worth tracking is cost per accepted change, not the token count. A loop without a hard gate does not fail loudly, it quietly runs up the bill (the so-called Ralph Wiggum loop: the agent calls the task done too early). Every serious loop needs a stop condition and an iteration limit.
---	---

7 Case study: a loop you can run yourself

Time to come down to earth, because things explain best on something that runs with a single command. Let's pick a task the model doesn't know from memory, because we draw it at random every time: **subset-sum**. You have a set of numbers and a target. Find a subset that sums exactly to the target. Sounds innocent, but it is one of the classic hard problems, NP-complete: **finding a solution can be brutally hard, but checking a given candidate is trivially easy**. That gap between "find" and "check" is the generator-verifier asymmetry, and it is exactly what explains why the gate must stand outside the model.

✓	BEST PRACTICE The whole codebase is open. The loop, the verifiers, the five problems and the benchmark from section 8 live in the repository github.com/blablast/agentic-loop.
---	---

You clone it, run "uv sync" and run "uv run python examples/run.py --problem subset_sum". There is also an offline variant, with no API key, so you can give students something that actually works in class, not just a slide.

Step 1: the verifier, i.e. the gate

The verifier is short, deterministic and has no idea what the model "thinks". It checks two things: whether the candidate is made of numbers from the set and whether it sums to the target. When it doesn't, it returns a concrete hint (e.g. "difference -6"), which the next attempt will use.

```
def verify(numbers, target, candidate):
    if any(x not in numbers for x in candidate):
        return False, "element spoza zbioru"
    if (total := sum(candidate)) != target:
        return False, f"różnica {target - total:+d}"
    return True, "OK"
```

listing 1 - the gate: returns true/false plus a concrete hint.

And here's the point. `verify([8, 11, 18, 27], 64)` returns `(True, "OK")`, but a likely wrong shot `verify([6, 8, 11, 18, 27], 64)` returns `(False, "difference -6")` – instantly, no argument. The model can be a hundred percent sure it is right. The gate still won't let a bad answer through. The model's confidence alone doesn't give you that (Huang et al., 2024).

Step 2: the loop, i.e. the maker proposes, the gate decides

The loop just ties it together: it asks the maker for a candidate, hands it to the gate, and when the gate refuses, it records the hint and asks for another attempt. You can see all four elements from section 1 at once: state, verification, iteration and a double stop condition.

```
def loop(numbers, target, maker, max_iters=40):
    history = [] # STATE: attempts
    for _ in range(max_iters): # STOP: hard limit
        candidate = maker(numbers, target, history)
        ok, feedback = verify(numbers, target, candidate)
        if ok:
            return candidate # STOP: success
        history.append((candidate, feedback))
    return None
```

listing 2 - the loop skeleton: all its power comes from a trusted gate.

Step 3: the maker on the Gemini model (and why without MCP)

The maker can be anything that proposes candidates, even plain search. In practice we plug in a model. Here Gemini: we put the gate's hints into the prompt of the next attempt, so the model learns from its own mistakes on the fly. And one important thing: **we don't need an MCP server for this**. The verifier is not a tool the model calls, it is an external test in our code. That's exactly the point: the gate is meant to be independent of the model.

```
from google import genai
from google.genai import types
```



```
def gemini_maker(numbers, target, history):
    client = genai.Client() # reads GEMINI_API_KEY
    tries = "\n".join(f"{c} -> {fb}" for c, fb in history)
    prompt = f"S={numbers}, target={target}. Tried:\n{tries}"
    resp = client.models.generate_content(
        model="gemini-3.5-flash",
        contents=prompt,
        config=types.GenerateContentConfig(
            thinking_config=types.ThinkingConfig(
                thinking_level="high"),
        )
    )
    return parse_list(resp.text) # e.g. [3, 7, 12]
```

listing 3 – the maker on Gemini (gemini-3.5-flash), key from GEMINI_API_KEY.

A good start is gemini-3.5-flash with thinking_level "high" – fast and cheaper, and the loop makes many calls. When a harder case shows up, you reach for the stronger gemini-3.1-pro. The rest of the code doesn't change, you only swap the maker.

The key point: search, heuristic and Gemini are interchangeable, because the gate decides success, not the author of the solution. This is exactly the maker/checker pattern. And that's why coding is the flagship task for agents: tests give a free, objective gate.

Four rules to close

1. Start with the simplest solution. An agent is a last resort, not a default.
2. A loop only makes sense when something can automatically reject a bad result.
3. Verification always external. The model's confidence is not the same as correctness.
4. Watch the cost per accepted change and always set a hard stop condition.

8 Experiment results: a benchmark sweep

The study in section 7 showed the loop on one example. But since the whole claim is "the gate, not the author, decides success", it deserves to be measured. Instead of a single run I fired off a **sweep**: the same loop skeleton and the same external verifier, but across a grid of models, problems and random seeds. In total **800 runs** – 9 models × 5 problems × 2 "thinking" levels (high / minimal) × 10 seeds, with a hard limit of 12 iterations. All models are local (Ollama), so we compare the loop's capability, not API prices. Each problem has a different candidate shape and a different, deterministic gate. Subset-sum from the study is just one of the five.

NOTE

i

The experiment plan. Five problems (subset_sum, optimization, regex_golf, reverse_engineering, maze), nine local models (from lfm2.5 and qwen3:4b up to gemma4:31b and deepseek-r1:32b), 10 seeds per cell, max 12 iterations. We measure three hardware-independent metrics: the share of solutions (succ%), the median number of iterations needed for success, and the median tokens used. We deliberately skip generation time, it depends on momentary GPU load, not on the loop's quality.

What the models can solve: success rate by problem

First, the difficulty. Averaged over all models and settings, the five problems form a clear ladder – from the almost-always-solvable subset-sum to the nearly unsolvable optimization and reverse_engineering.

Problem	Success rate (all models)	Nature
subset_sum	93.1%	candidate trivial to check – the floor problem
regex_golf	73.8%	short space, numeric gate – easy
maze	37.5%	path planning – medium, strongly model-dependent
reverse_engineering	10.6%	hidden function – hard, narrow feedback
optimization	8.8%	continuous loss minimization – the hardest

Here's the first conclusion that's easy to miss: **subset-sum from the case study is the easiest of the whole set** – with "high" thinking every one of the nine models solves it 100% of the time. Good for teaching, but the real test of the loop is optimization and reverse_engineering, where almost everything fails. And here comes the most important thing in the whole material.

BEST PRACTICE



The gate holds even where the model loses. On optimization and reverse_engineering the models generated tens of thousands of correctly parsed candidates and got concrete feedback ("loss 19.1", "wrong input -7"), and yet the verifier did not let a single bad answer through. The loop does not "give up" and does not cheat: it guarantees **correctness, not capability**. A weak model does not become smarter, it merely becomes unable to pass off a wrong result as a good one. This is exactly the generator-verifier asymmetry from section 7, confirmed on 1600 runs.

Does iteration really help: feedback in numbers

If the loop were just "retrying", successes would always land on the first shot. Instead, of 358 successful runs **62% succeed right away, but 38% only after at least two iterations** – with a tail reaching the full 12 tries. More than a third of the wins are the direct result of the gate's hints coming back into the prompt: Reflexion from section 4 in action, not on a slide.

Stronger model or more thinking?

Turning on "high" mode raises the average success rate from 37% to 52% (on models that have both modes). Thinking helps, but not monotonically. gpt-oss on reverse_engineering **loses by thinking** (80% → 40%): the classic case of overthinking. Scaling size also gives surprisingly little – deepseek 14b→32b only 42%→46%, and qwen3 4b→14b 40%→46%. On these tasks the outcome is decided by the gate and iteration rather than the parameters themselves.



CAUTION

More power is neither free nor certain. A model twice the size is marginally better, and more "thinking" can hurt (gpt-oss). At the same time token cost differs about twentyfold between models: the median per successful run ranges from ~800 tokens (l1m2.5) to ~9700 (gemma4:26b). That's why the metric is **cost per accepted result**, not the raw success rate – exactly as in section 6.

Table: model ranking – success rate across all problems and median tokens per success.

Model	Success rate (all problems)	Median tok. / success
gpt-oss:latest	74%	3914
gemma4:31b	61%	1616
gemma4:26b	60%	3052
deepseek-r1:32b	46%	2328
deepseek-r1:14b	42%	2275
qwen3:4b	39%	4474
qwen3:14b	31%	2217
gemma4:latest	29%	1387
l1m2.5:latest	20%	787

Full results by problem

Below is the full set of 80 cells (problem × model × mode), 10 seeds each. The column **succ%** is the share of solutions, **median iter.** is the median number of iterations, where an unsolved run counts as 12, and **median tok.** is the median total tokens per run. I ran deepseek in "high" mode only.

Problem: subset_sum

Model	Thinking	succ%	median iter.	median tok.
gpt-oss:latest	high	100	1	521.0
gpt-oss:latest	minimal	100	1	548.0
gemma4:31b	high	100	1	1 050.0
gemma4:31b	minimal	100	1	294.5
gemma4:26b	high	100	1	1 846.0
gemma4:26b	minimal	100	1	340.5
deepseek-r1:32b	high	100	1	1 640.5
deepseek-r1:14b	high	100	1	1 427.0
qwen3:4b	high	100	1	3 051.5
qwen3:4b	minimal	100	1	2 549.5
qwen3:14b	high	100	1	1 889.0
qwen3:14b	minimal	40	12	5 909.5

gemma4:latest	high	100	1	1 124.5
gemma4:latest	minimal	50	11	4 064.0
lfm2.5:latest	high	100	1	787.0
lfm2.5:latest	minimal	100	1	778.5

Problem: optimization

Model	Thinking	succ%	median iter.	median tok.
gpt-oss:latest	high	0	12	12 447.5
gpt-oss:latest	minimal	20	12	13 167.5
gemma4:31b	high	50	11.5	15 831.5
gemma4:31b	minimal	0	12	3 715.0
gemma4:26b	high	40	12	37 089.0
gemma4:26b	minimal	20	12	3 686.0
deepseek-r1:32b	high	0	12	23 985.5
deepseek-r1:14b	high	0	12	17 215.0
qwen3:4b	high	0	12	44 660.0
qwen3:4b	minimal	0	12	82 228.5
qwen3:14b	high	0	12	21 138.0
qwen3:14b	minimal	0	12	3 884.0
gemma4:latest	high	10	12	11 941.0
gemma4:latest	minimal	0	12	3 684.5
lfm2.5:latest	high	0	12	12 557.5
lfm2.5:latest	minimal	0	12	47 767.5

Problem: regex_golf

Model	Thinking	succ%	median iter.	median tok.
gpt-oss:latest	high	100	1	1 854.5
gpt-oss:latest	minimal	100	1	2 284.5
gemma4:31b	high	100	1	1 686.5
gemma4:31b	minimal	90	1	261.0
gemma4:26b	high	100	1	5 411.5
gemma4:26b	minimal	90	2	989.0
deepseek-r1:32b	high	90	1	2 981.0
deepseek-r1:14b	high	100	1	3 042.5
qwen3:4b	high	90	1	8 082.0
qwen3:4b	minimal	90	1	6 793.0

qwen3:14b	high	80	1.5	6 459.0
qwen3:14b	minimal	40	12	6 105.0
gemma4:latest	high	90	1	2 192.0
gemma4:latest	minimal	20	12	6 829.0
lfm2.5:latest	high	0	12	23 200.0
lfm2.5:latest	minimal	0	12	37 440.0

Problem: reverse_engineering

Model	Thinking	succ%	median iter.	median tok.
gpt-oss:latest	high	40	10.5	116 368.0
gpt-oss:latest	minimal	80	5.5	82 334.5
gemma4:31b	high	10	12	104 214.0
gemma4:31b	minimal	0	12	13 578.5
gemma4:26b	high	40	12	250 312.0
gemma4:26b	minimal	0	12	63 436.0
deepseek-r1:32b	high	0	12	125 662.0
deepseek-r1:14b	high	0	12	137 346.0
qwen3:4b	high	0	12	188 898.0
qwen3:4b	minimal	0	12	192 484.0
qwen3:14b	high	0	12	141 372.0
qwen3:14b	minimal	0	12	13 065.0
gemma4:latest	high	0	12	64 591.5
gemma4:latest	minimal	0	12	12 444.5
lfm2.5:latest	high	0	12	800 701.0
lfm2.5:latest	minimal	0	12	762 882.0

Problem: maze

Model	Thinking	succ%	median iter.	median tok.
gpt-oss:latest	high	100	7	20 070.5
gpt-oss:latest	minimal	100	7	19 203.0
gemma4:31b	high	100	7	14 154.0
gemma4:31b	minimal	60	11	5 252.0
gemma4:26b	high	100	7	37 048.0
gemma4:26b	minimal	10	12	6 233.5
deepseek-r1:32b	high	40	12	55 624.5
deepseek-r1:14b	high	10	12	48 622.0

qwen3:4b	high	10	12	118 324.0
qwen3:4b	minimal	0	12	140 962.0
qwen3:14b	high	50	10.5	51 003.5
qwen3:14b	minimal	0	12	5 601.0
gemma4:latest	high	20	12	35 730.5
gemma4:latest	minimal	0	12	5 545.0
lfm2.5:latest	high	0	12	47 469.5
lfm2.5:latest	minimal	0	12	70 080.0

The conclusion ties the whole material together: the results confirm the claim from section 5 with hard numbers. The loop did not turn a weak model into a strong one, it turned it into a model that **does not pass off a wrong answer as a correct one**, and where it could, iterative feedback really shortened the road to a solution. An external, objective gate and a hard stop condition turn out to matter more than model size or the amount of "thinking" – and that's the lesson worth keeping, for yourself and for others, before the section on loops that improve instructions themselves.

9 What's new: a loop that improves its own skill

So far the loop solved a task. But what if we point it at the very instructions we feed the agent? This is the freshest current in the field (2025–2026) and worth writing about, because that is exactly where the engineer's role is shifting.

Anyone who has ever written an instruction for an agent (a so-called **skill** – a file with a procedure) knows the pain: you fix one thing, break another, and don't know which line does what. Tweaking this text by hand doesn't scale. New frameworks propose something that sounds bold but works: let's treat the skill file as **trainable state** and improve it with a loop, without touching the model's weights.

SkillOpt: training the instruction file

SkillOpt (Microsoft, open-source) does it with the discipline of network training. Over and over: it runs the agent on a set of tasks and collects the runs. A second optimizer model reads what failed, proposes small edits to the skill (add/remove/replace), bounded by a "textual learning rate" so as not to wreck the whole thing. And then the gate: it accepts a change only if it really improves the result on a separate test set. Rejected edits go into a buffer so mistakes don't repeat. The effect: the skill only gets better, never worse, and you end up with a small file that costs nothing at run time. Across 52 tested configurations it was best or tied for best, and on GPT-5.5 it raised success by more than 23 points.

NOTE

i

The SkillOpt loop: rollout (run the tasks) → reflection (what failed) → bounded text edits → held-out gate (accept only if it improves) → buffer of rejects. These are the familiar phases of verification and iteration, only applied to the instruction itself, not to the task.

GEPA: evolution through reflection

GEPA (Genetic-Pareto) approaches this from the evolutionary side. It keeps several prompt versions at once, reads execution traces, diagnoses in plain language what went wrong, and picks the best variants from a so-called Pareto front. Importantly, it does this frugally – in an ICLR 2026 paper it beats reinforcement learning with about 35 times fewer runs – and it works with the popular DSPy framework. EvoSkill and TextGrad are related ideas, among others for coding-agent skills.

CAUTION

!

This is not magic. These systems only work where a verifiable signal and a clean held-out set exist – the same conditions every good loop requires. On subjective, open-ended tasks they won't work. You pay the cost up front (the optimizer reads a lot of traces), but the finished skill no longer burdens inference.

Notice the punchline: it is the same idea that runs through the whole material, just one floor up. **Even self-improvement is safe only behind an objective, external gate.** For an educator that's an important signal: the engineer's role shifts from tinkering with single sentences in a prompt to designing systems – with metrics, memory and stop conditions. And that is exactly what's worth showing students today.

► WHAT TO REMEMBER

An agentic loop is an LLM using tools in a cycle of plan – act – verify – iterate. Its core has a peer-reviewed lineage (ReAct, Reflexion, Self-Refine), and one condition decides everything. Verification must be external, because a model's confidence is not correctness. The subset-sum study shows this tangibly, because finding a solution is hard and checking a candidate is easy. You don't build a loop for everything. What matters is repeatability, an objective gate and a hard stop condition, and cost grows fast. The newest current (SkillOpt, GEPA) shows the same loop can be pointed at the agent's own instructions and improve them automatically, also only behind an external gate. If you teach this to others or build a product with it, this is probably the single most important thing to remember: a good loop is not a smarter model, it is a better gate.

Sources

Peer-reviewed / arXiv

Wei J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 (NeurIPS 2022).

Yao S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629 (ICLR 2023).

Yao S. et al. (2023). Tree of Thoughts: Deliberate Problem Solving with Large Language Models. arXiv:2305.10601 (NeurIPS 2023).

Shinn N. et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. arXiv:2303.11366 (NeurIPS 2023).

Madaan A. et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 (NeurIPS 2023).

Gou Z. et al. (2024). CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. arXiv:2305.11738 (ICLR 2024).

Dhuliawala S. et al. (2023). Chain-of-Verification Reduces Hallucination in Large Language Models. arXiv:2309.11495 (ACL Findings 2024).

Huang J. et al. (2023). Large Language Models Cannot Self-Correct Reasoning Yet. arXiv:2310.01798 (ICLR 2024).

Stechly K., Valmeekam K., Kambhampati S. (2024). On the Self-Verification Limitations of LLMs on Reasoning and Planning Tasks. arXiv:2402.08115.

Agrawal L. et al. (2025). GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. arXiv:2507.19457 (ICLR 2026, oral).

Microsoft Research (2026). SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904 (open-source: github.com/microsoft/SkillOpt).

Lab guidelines

Anthropic (2024). Building Effective Agents.

Anthropic (2025). Effective Context Engineering for AI Agents. Effective Harnesses for Long-Running Agents. How We Built Our Multi-Agent Research System.

OpenAI (2025). A Practical Guide to Building Agents.

Practice / thought leaders

Ng A. (2024). Four Agentic Design Patterns (The Batch, DeepLearning.AI).

Huntley G. (2025). The Ralph Wiggum Loop – external validation and fresh context.

Cherny B. / Anthropic (2025). Claude Code: Best Practices for Agentic Coding (research → plan → execute → review → ship).

Strus B. (2026). Agentic loop – a working loop, verifiers, five problems and the benchmark from section 8. Open source: github.com/blablast/agentic-loop.